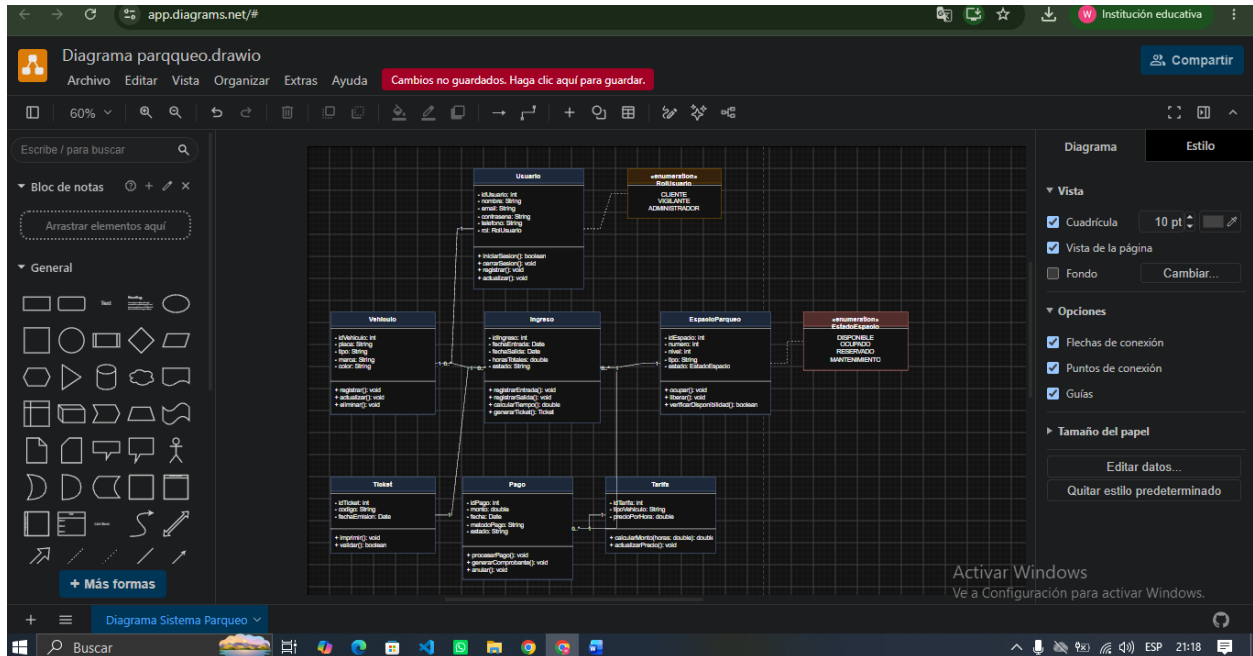
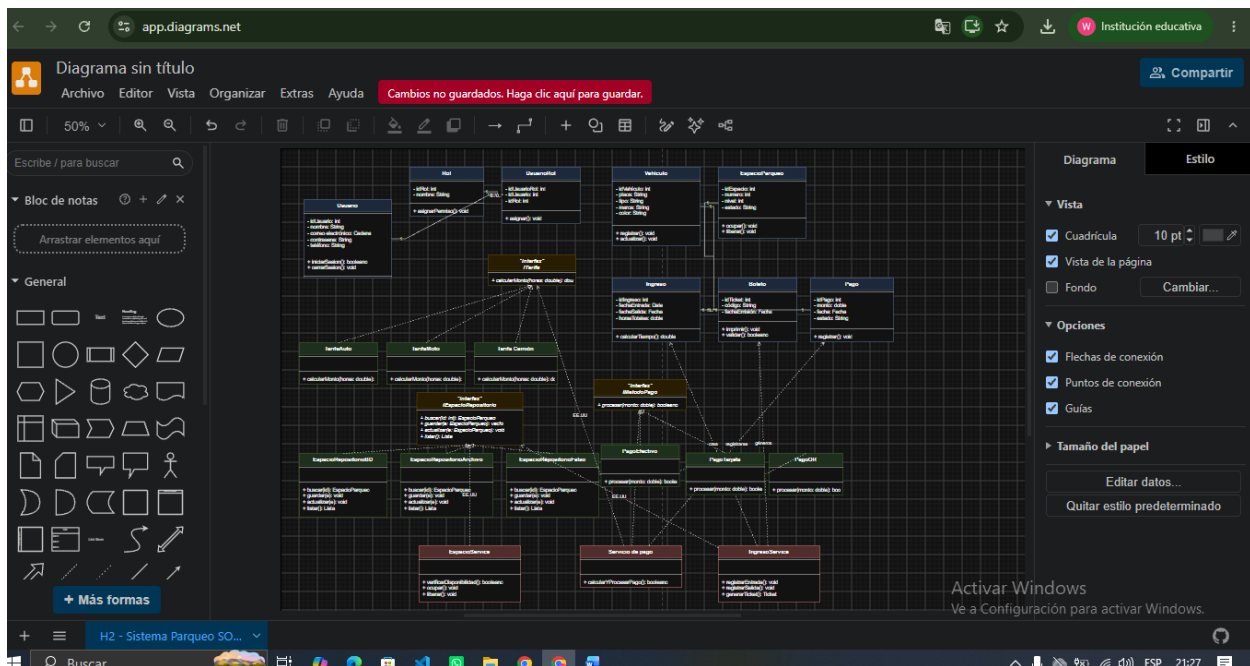


Aplicación de SOLID (ANTES Y DESPUES)



H1 - ANTES

- En este primer diseño representé las clases principales del sistema de parqueo: Usuario, Vehículo, Ingreso, EspacioParqueo, Ticket, Pago y Tarifa.
- La clase Pago tenía varias responsabilidades: guardaba el método de pago como un simple texto (metodoPago: String) y su lógica de procesamiento dependía de revisar ese texto internamente.
- La clase Tarifa concentraba el cálculo del monto para todos los tipos de vehículo en un solo método, sin distinguir claramente la lógica de cada tipo.
- El rol del usuario (RolUsuario) era solo un dato dentro de Usuario, sin una estructura propia para manejar permisos.
- El acceso a los datos de EspacioParqueo no tenía ninguna capa de abstracción: cualquier cambio en la forma de almacenar los espacios afectaría directamente a las demás clases.
- También existían relaciones directas entre Vehículo, Ingreso, EspacioParqueo, Ticket y Pago, sin interfaces que separaran responsabilidades.
- El diseño no tenía interfaces para controlar las dependencias ni para permitir agregar nuevas variantes (tipos de tarifa, métodos de pago) sin modificar código existente.
- Por estas razones, este diseño necesitaba ser mejorado aplicando los principios SOLID.



H2 - DESPUÉS

- En este segundo diseño mejoré el sistema separando las responsabilidades y agregando nuevas clases para que cada parte tenga una función más clara.
- Incorporé Rol y UsuarioRol para controlar el acceso al sistema, en vez de mantener el rol como un simple dato dentro de Usuario (SRP).
- Mantuve Vehiculo, EspacioParqueo, Ingreso, Ticket y Pago, conservando sus relaciones porque representan las operaciones principales del parqueo.
- Separé el cálculo de tarifas mediante ITarifa y las clases TarifaAuto, TarifaMoto y TarifaCamion, evitando depender de una sola lógica con condicionales (OCP).
- Separé los métodos de pago mediante IMetodoPago y las clases PagoEfectivo, PagoTarjeta y PagoQR, permitiendo agregar un nuevo método sin modificar el existente (OCP/ISP).
- También agregué EspacioService, IngresoService y Servicio de pago (PagoService) para organizar las operaciones de cada parte del sistema (SRP).
- Incorporé IEspacioRepositorio con tres implementaciones (BD, Archivo y Falso) para que el sistema no dependa directamente de una única forma de guardar los espacios (DIP).
- Con estas mejoras aplico principalmente SRP, OCP, ISP y DIP, haciendo que agregar cambios sea más sencillo sin afectar todo el sistema.
- El resultado es un diseño más organizado, relacionado y preparado para posteriormente implementar el sistema.